Day 14: Weekly Revision

1. Day Number

Day 14

2. Topic Name

Revision of Days 8–13

Today you will revise:

- Menu using `if-elif-else`
- Repeating menu using a `while` loop
- Printing options using a `for` loop and `range()`
- Storing transactions in a list
- Reading list items using a loop
- Organizing deposit logic inside a function
- Returning the updated balance

---

3. Connection

During Days 8–13, you learned each concept separately.

Today you will combine them into one small bank program:

```
Menu
  ↓
User chooses deposit
  ↓
Deposit function checks amount
  ↓
Balance is updated
  ↓
Transaction is saved in a list
  ↓
Menu appears again
  ↓
Program stops when user chooses exit
```

This revision combines **menu, loop, list, and one function**.

---

4. Revision Summary of Days 8–13

Day 8: Simple Menu Choice

You displayed options and used `if-elif-else` to check the user's choice.

Example idea:

```
1. Deposit
2. Exit
```

Day 9: Repeat Menu Using `while`

You used a `while` loop to show the menu repeatedly.

The loop continued until the user selected the exit option.

Day 10: Print Numbered Options Using `range()`

You stored menu options in a list and printed them using a loop.

Example:

```
options = ["Deposit", "Exit"]
```

The index helped create option numbers such as `1` and `2`.

Day 11: Store Transaction in a List

You created an empty transaction list:

```
transactions = []
```

After a successful deposit, you added a message using `append()`.

Day 12: Show Transactions Using a Loop

You used a `for` loop to read and print every transaction stored in the list.

You also learned to check whether the list was empty.

Day 13: Deposit Function

You moved the deposit validation and balance update into a function.

The function accepted:

- Current balance
- Deposit amount

It returned:

- Updated balance when the amount was positive
- The same balance when the amount was invalid

---

5. Important Topics

Menu

A menu gives the user a list of actions.

```
1. Deposit Money
2. Exit
```

`while` Loop

A `while` loop repeats code while its condition remains `True`.

It is useful when you do not know how many times the user will use the menu.

`for` Loop

A `for` loop reads items one by one.

It can be used to:

- Print menu options
- Print transaction history

`range()`

`range()` generates a sequence of numbers.

It is helpful when you need list indexes.

List

A list stores multiple values.

```
transactions = []
```

Function

A function groups related logic into one reusable block.

```
def deposit_money(...):
```

`return`

`return` sends a result back to the place where the function was called.

For this program, the function should return the balance.

---

6. Foundational Notes

Keep balance outside the function

The main program owns the current balance.

The function receives the balance, checks the deposit amount, and returns the result.

```
Main program gives balance to function
                  ↓
Function calculates updated balance
                  ↓
Function returns balance to main program
```

Save only successful deposits

A deposit should be saved in the transaction list only when the amount is greater than `0`.

Do not save:

- Negative deposits
- Zero deposits
- Invalid menu choices

Use `break` for exit

When the user chooses the exit option, use `break` to stop the `while` loop.

An empty list is not an error

At the beginning:

```
transactions = []
```

This is completely valid.

It only means no successful deposit has happened yet.

Update the balance using the returned value

Calling the function is not enough.

The returned balance must be stored again.

Conceptually:

```
balance = deposit_function(balance, amount)
```

---

7. Easy Example

This example shows a function that adds a positive number to a total. It is not the complete revision solution.

```
def add_points(current_points, new_points):
    if new_points > 0:
        return current_points + new_points

    return current_points


<div class="source-blank-line"></div>

points = 10
points = add_points(points, 5)

print(points)
```

What happens?

1. `points` starts at `10`.
2. The function receives `10` and `5`.
3. Because `5` is positive, it adds the values.
4. The function returns `15`.
5. The returned value is saved back in `points`.

Output:

```
15
```

---

8. Revision Problem Statement

Create a simple bank program with a menu that repeats until the user chooses exit.

The menu should contain only two options:

```
1. Deposit Money
2. Exit
```

Program requirements:

- Create an initial balance.
- Create an empty transaction list.
- Display the menu repeatedly.
- Ask the user to select an option.
- When the user chooses deposit:
- Ask for the deposit amount.
- Use a function to validate the amount.
- Update the balance only when the amount is positive.
- Save every successful deposit in the transaction list.
- When the user chooses exit:
- Stop the menu loop.
- Display the final balance.
- Display the successful transactions.
- Show an appropriate message for an invalid menu choice.

---

9. Concepts Used

Concept	**Purpose**
Variable	Store balance and user input
String	Store menu and transaction messages
Number	Store deposit amount and balance
List	Store successful deposits
`append()`	Add a transaction to the list
`while`	Repeat the menu
`break`	Stop the menu
`if-elif-else`	Handle menu choices
`for`	Print menu options or transactions
`range()`	Generate menu option numbers
Function	Organize deposit logic
Parameter	Pass balance and amount into the function
`return`	Send the balance back
Comparison operator	Check whether the amount is positive

10. Thought Process

Step 1: Decide the starting data

The program needs:

- An initial balance
- A list of menu options
- An empty transaction list

Example thinking:

```
balance = starting value
menu options = Deposit and Exit
transactions = empty list
```

Step 2: Create the deposit function

The function should accept:

```
current balance
deposit amount
```

Inside the function:

- Check whether the amount is greater than `0`.
- If valid, return the updated balance.
- Otherwise, return the original balance.

Step 3: Start the menu loop

Use a `while` loop because the menu must continue until the user chooses exit.

Step 4: Display the menu

You can print the options directly or use a `for` loop with `range()`.

Remember that list indexes start from `0`, but menu numbers normally start from `1`.

Step 5: Read the user's choice

The user should select either:

```
1
```

or:

```
2
```

Step 6: Handle the deposit option

When the user selects deposit:

1. Ask for the amount.
2. Check whether it is positive.
3. Call the deposit function.

4. Store the returned balance.
5. Add a transaction message only when the deposit succeeds.

Step 7: Handle the exit option

When the user selects exit:

- Print an exit message.
- Use `break`.

Step 8: Handle invalid choices

Any choice other than the available menu options should show an invalid-choice message.

The loop should continue afterward.

Step 9: Show transaction history

After the loop ends:

- Check whether the transaction list is empty.
- If it is empty, show that no transactions were found.
- Otherwise, print each transaction using a `for` loop.

---

11. Pseudocode

```
START

CREATE a deposit function with balance and amount

    IF amount is greater than zero
        ADD amount to balance
        RETURN updated balance
    ELSE
        RETURN original balance

SET starting balance

CREATE menu options list
CREATE empty transaction list

START an infinite while loop

    PRINT menu options using a loop

    ASK user for menu choice

    IF choice is deposit

        ASK user for deposit amount

        IF deposit amount is greater than zero

            CALL deposit function
            SAVE returned balance
            ADD successful deposit message to transaction list
            PRINT updated balance

        ELSE

            PRINT invalid deposit message

    ELSE IF choice is exit

        PRINT exit message
        BREAK the loop

    ELSE

        PRINT invalid menu choice

PRINT final balance

IF transaction list is empty

    PRINT no transactions found

ELSE
```

```
    PRINT transaction heading

    FOR each transaction in transaction list
        PRINT transaction

END
```

---

12. Suggested Solving Approach: Functional Approach

Keep the deposit calculation inside one function.

Suggested responsibility:

```
deposit function:
    receives balance and amount
    validates the amount
    returns the balance
```

Suggested main-program responsibility:

```
main program:
    displays menu
    reads input
    calls function
    saves transactions
    controls the loop
    prints final results
```

This separation makes the program easier to:

- Read
- Test
- Debug
- Extend later

---

13. Easy Edge Cases

Invalid Menu Choice

Input:

```
5
```

Expected behaviour:

```
Invalid menu choice
```

The program should show the menu again.

Negative Deposit

Input:

```
-100
```

Expected behaviour:

```
Deposit amount must be positive
```

The balance should not change, and no transaction should be saved.

Deposit Amount `0`

Input:

```
0
```

Expected behaviour:

```
Deposit amount must be positive
```

Zero is not a successful deposit.

Empty Transaction List

The user selects exit without making a successful deposit.

Expected behaviour:

```
No transactions found
```

Multiple Successful Deposits

The user deposits more than once.

Every successful deposit should be stored separately in the transaction list.

---

14. Common Mistakes to Avoid

Forgetting to update the returned balance

Incorrect idea:

```
deposit_money(balance, amount)
```

The function may return a new balance, but it is not being saved.

Remember to assign the returned value.

Saving invalid deposits

Do not add negative or zero deposits to the transaction list.

Putting `break` in the wrong place

`break` should run only when the user chooses exit.

If it is placed inside the deposit option, the program may stop after one deposit.

Using the wrong menu numbering

List indexes begin at `0`, but users normally expect options to begin at `1`.

Forgetting `append()`

A transaction will not be saved unless it is added to the list.

Returning too early

Do not place `return` before the deposit validation and balance calculation are complete.

Printing the empty list directly

Printing:

```
[]
```

is less user-friendly than displaying:

```
No transactions found
```

Creating the transaction list inside the loop

If you create the empty list inside the `while` loop, old transactions will be erased every time the menu repeats.

Create it before the loop.

---

15. Quick Self-Check Questions

1. Why is a `while` loop suitable for this menu?
2. What is the difference between `print()` and `return`?
3. Why should the transaction list be created before the loop?
4. When should a deposit be added to the transaction list?
5. What happens if the returned balance is not assigned back to the balance variable?

---

16. Hint Only

Start with these main parts:

```
def deposit_money(balance, amount):
    # Validate the amount
    # Return updated or original balance
```

Then prepare the program data:

```
balance = ...
menu_options = [...]
transactions = []
```

Use this overall structure:

```
while True:
    # Print menu using range()

    # Read choice

    if ...:
        # Ask for deposit
        # Call function
        # Save successful transaction

    elif ...:
        break

    else:
        # Invalid choice
```

After the loop, check the list:

```
if not transactions:
    # No transactions
else:
    # Print transactions one by one
```

Focus especially on these two rules:

```
Save the balance returned by the function.
Append a transaction only after a successful deposit.
```